

Exercises

Function Application and Function Composition

Exercise 1:

Write **bigCubes** that takes a list and returns a list of cubes that are > 500

```
bigCubes :: [Int] -> [Int]
```

Solution 1:

```
bigCubes :: [Int] -> [Int]
bigCubes xs = filter (>500) $ map (^3) xs
```

Exercise 2:

Write **lottaBiggest** that takes a list and replicates the largest element 4 times.

```
lottaBiggest :: [Int] -> [Int]
e.g.
lottaBiggest [2,5,3,1] = [5,5,5,5]
```

Hint: You can use the *maximum* function that returns the maximum value in a numeric list.

Solution 2:

Using \$:

```
lottaBiggest :: [Int] -> [Int]
lottaBiggest xs = replicate 4 $ maximum xs
```

or (using function composition)

```
lottaBiggest' :: [Int] -> [Int]
lottaBiggest' = replicate 4 . maximum
```

Exercise 3:

Write **powers** that takes a number and creates a list of that number squared, cubed, and quadrupled.

e.g. `powers 2 = [4,8,16]`

Solution 3:

```
powers :: Int -> [Int]
powers x = map ($x) [(^2),(^3) ,(^4) ]
```

Exercise 4:

Given **process** that takes a list of numbers, filters out the negative numbers, doubles the remaining numbers, and returns the sum of the doubled numbers:

```
process x = sum (map double (filter (>0) x))
```

Rewrite **process** using function composition and eliminating the x parameter.

Solution 4:

```
process :: [Int] -> Int
process = sum . map double . filter (>0)
```

Exercise 5:

Given

```
compute x y = length (take x (filter even y))
```

which takes a number and a list of numbers, filters out the even numbers, and returns the length of the first x even numbers. Rewrite **compute** using function composition and eliminating the y parameter.

Solution 5:

```
compute :: Int -> [Int] -> Int
compute x = length . take x . filter even
```

Exercise 6:

Assume people are dining. We have a list of tip percents (assume people tip at different rates):

```
e.g. pcts = [0.15, 0.2, 0.21]
```

We have a list of bills (what people owe, minus tip)

```
e.g. amts = [20.5, 30, 25]
```

Write **calcBill** that takes `amts` and `pcts` and calculates what each person will pay, based on their amt and pct. Then apply a 4% tax rate.

```
calcBillamts pcts :: [Float] -> [Float] -> Float
calcBillamts pcts [20.5, 30, 25] [0.15, 0.2, 0.21] = [24.518,37.44,31.46]
```

Solution 6:

```
calcBillamts pcts :: [Float] -> [Float] -> [Float]
calcBillamts pcts amts tips =map (*1.04) $ zipWith (+) amts $
                                zipWith (*) amts tips
-- rightmost calculates tips , next adds tips, then add 4%
```

Exercise 7:

What does the following function do and how does it work?

```
clamp :: Double -> Double
clamp = max 0 . min 1
```

Solution 7:

clamp takes a number and clamps it to the range $[0,1]$. It does this by first applying *min 1* to the input, which ensures that the result is at most 1. Then it applies *max 0* to that result, which ensures that the final output is at least 0. Thus, any input less than 0 will return 0, any input greater than 1 will return 1, and any input between 0 and 1 will return itself.